Lab Activity: Collaborating with Git and GitHub

What is a version control system?

A **version control system (VCS)** is software that tracks changes to files over time. It lets you see who changed what and when, go back to any earlier version, and lets several people work on the same project without overwriting each other's work. It is used mostly for source code, but works for any text-based files.

Instead of keeping one current copy of a project, a VCS stores a history of **snapshots** called commits. Each commit records what changed, who made the change, when it was made, and a message explaining why.

Types of version control systems

1. **Local VCS:** keeps version history on one machine only (e.g. RCS). Largely obsolete.
2. **Centralized VCS:** one central server holds the history and clients check files out from it (e.g. SVN, CVS). Simple, but the server is a single point of failure and committing needs a network connection.
3. **Distributed VCS:** every developer has a full copy of the repository and its history (e.g. **Git**, Mercurial). You can work offline, branching and merging are fast, and there is no single point of failure.

Git is today's dominant VCS. It is usually paired with a hosting platform such as **GitHub**, GitLab or Bitbucket, which adds pull requests, code review, issue tracking and automation on top.

In today's lab you will use **Git** with **GitHub**. The terms below come up throughout the activity.

Key concepts

Term	Meaning
Repository (repo)	The project plus its full change history
Remote / origin	The copy of the repository hosted on GitHub; `origin` is the default name Git gives it
Clone	Getting a full working copy of a repository
Working directory	The project files on your computer that you edit

Term	Meaning
Staging (`git add`)	Choosing which changes go into the next commit
Commit	A saved snapshot of changes, with a message
Commit hash	The unique code that identifies a commit, e.g. `a9b8c7d`
Branch	An independent line of development, e.g. for a new feature, that doesn't affect the main code until combined
`main` branch	The main line of the project that holds the working, approved code
Feature branch	A short-lived branch for one feature or fix, e.g. `feature/add-student`
HEAD	A pointer to the branch and commit you are currently on
Push / pull	Sending your commits to the shared repo / fetching others' commits from it
Merge	Combining changes from one branch into another
Merge commit	A commit that records where two branches were joined
Conflict	When two people change the same lines differently; Git asks a human to resolve it
Collaborator	A GitHub user given permission to push to someone else's repository
Branch protection	A rule that stops changes reaching a branch, such as `main`, without a reviewed pull request
Issue	A GitHub task, bug or feature request used to plan and track work
Pull request (PR)	A request on GitHub to merge a branch, where teammates review the changes first
Code review	Reading a teammate's PR and choosing Comment, Request changes or Approve
Revert	A new commit that undoes an earlier commit without deleting history

Term	Meaning
Tag	A permanent label on a specific commit, e.g. `v1.0`
Release	A GitHub page built on a tag that marks a finished version, with release notes
`.gitignore`	A file listing files Git should not track, e.g. `__pycache__/`
README	The file shown on the repository's front page that explains the project

Why it matters

- **Undo mistakes:** roll back to any working version.
- **Collaboration:** many developers work in parallel on separate branches.
- **Accountability:** the history shows who changed what and why, which helps with debugging.
- **Experimentation:** try ideas on a branch without risking the main code.
- **Backup:** with Git, every clone is a full copy of the project and its history.

This lab puts all of these ideas into practice on a small team project.

Overview

Pairs of students build a small Python "Grade Book" program together on GitHub, following the same workflow real software teams use: issues, feature branches, pull requests, code review, merge conflict resolution, and a tagged release.

Before the lab, each student must:

- [ ] Create a GitHub account
- [ ] Install Git and check it works: `git --version`
- [ ] Set up authentication to push from the terminal (a Personal Access Token, SSH key, or signing in through VS Code / GitHub Desktop)
- [ ] Configure their identity:

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Learning outcomes

By the end of the lab, students will be able to:

1. Create a shared repository, add a collaborator, and protect the `main` branch.
2. Plan work with GitHub Issues and link code changes to them.
3. Develop features on separate branches with small, well-described commits.
4. Open a pull request, review a partner's code, and respond to review comments.
5. Resolve a merge conflict locally and push the fix.
6. Keep a local copy in sync with the remote, and publish a tagged release.

Roles

Each pair picks one Maintainer and one Collaborator. Both write code; the Maintainer also owns the repository settings, like a team lead on a real project.

Role	Responsibilities
Maintainer (Student A)	Creates the repo, pushes the starter code, adds the partner as collaborator, protects `main`, creates the release. Builds the **Add student** feature.
Collaborator (Student B)	Accepts the invite, clones the repo, reviews the Maintainer's pull request. Builds the **Show grades** feature.

The rule for both: **nobody pushes directly to `main`.** Every change reaches `main` through a reviewed pull request.

Starter code

The Maintainer saves this as `grade_book.py`. It runs, but the menu only offers Exit; each student adds one feature.

```
# grade_book.py
# A simple command-line grade book.

students = {}  # name -> list of marks


def show_menu():
    print("\n=== Grade Book ===")
    print("0. Exit")
```

```
def main():
    while True:
        show_menu()
        choice = input("Choose an option: ")
        if choice == "0":
            print("Goodbye!")
            break
        else:
            print("Invalid option.")


if __name__ == "__main__":
    main()
```

Run it with `python grade_book.py`.

Activity phases

The activity runs in six phases. Each phase ends with a checkpoint the instructor can verify before the pair moves on.

Phase 1: Set up the shared repository (Maintainer)

1. On GitHub, create a new repository named `grade-book`. Make it **Public**, tick **Add a README file**, and choose the **Python** `.gitignore` template.
2. Clone it and move into the folder:

```
git clone https://github.com/<maintainer-username>/grade-book.git
cd grade-book
```

3. Add `grade_book.py` (starter code above), then commit and push it. This is the **only** direct push to `main` in the whole activity.

```
git add grade_book.py
git commit -m "Add starter grade book with menu loop"
git push origin main
```

4. Go to **Settings → Collaborators → Add people** and invite your partner by username.
5. Go to **Settings → Branches → Add branch protection rule** (newer accounts may show **Rules → Rulesets**). For branch `main`, tick **Require a pull request before merging** and set **Required approvals** to 1.

Collaborator: accept the invitation (from email or the repository page), then clone the repository the same way.

> Checkpoint 1: both students run `git log --oneline` and see the starter commit.

Both students run `git log --oneline` on `main`. Expected output:

```
b7e3a1c (HEAD -> main, origin/main, origin/HEAD) Add starter grade book with
menu loop
8743cf3 Initial commit
```

The commit hashes (the 7-character codes) will be different on your computer. Check the commit messages and the labels in brackets.

Phase 2: Plan the work with Issues

Real teams start every change from an issue. The Maintainer creates three issues under the **Issues** tab:

Issue	Title	Assignee	Label
#1	Add a student with marks	Maintainer	enhancement
#2	Show each student's grade	Collaborator	enhancement
#3	Show the class average	Unassigned (bonus)	good first issue

Labels are coloured tags chosen from the **Labels** menu in the issue's right sidebar; GitHub provides them by default. `enhancement` marks a new feature, and `good first issue` marks a small, easy task that is open for anyone to pick up. Real open-source projects use it to point newcomers to a manageable first contribution.

> Checkpoint 2: three issues visible, #1 and #2 assigned.

Phase 3: Build features on branches (both, in parallel)

Each student creates their own branch from an up-to-date `main`:

```
git checkout main
git pull origin main
git checkout -b feature/add-student   # Maintainer
git checkout -b feature/show-grades   # Collaborator
```

Maintainer: Add student (Issue #1)

Add this function above `main()`:

```
def add_student():
    name = input("Student name: ")
    marks = input("Marks separated by spaces: ")
    students[name] = [int(m) for m in marks.split()]
    print(f"Added {name}.")
```

Collaborator: Show grades (Issue #2)

Add these functions above `main()`:

```
def calculate_grade(average):
    if average >= 85:
        return "A"
    elif average >= 70:
        return "B"
    elif average >= 55:
        return "C"
    elif average >= 40:
        return "D"
    return "F"


def show_grades():
    if not students:
        print("No students yet.")
        return
    for name, marks in students.items():
        average = sum(marks) / len(marks)
        print(f"{name}: average {average:.1f}, grade
{calculate_grade(average)}")
```

To test before the Add feature exists, temporarily add a line such as `students["Ali"] = [78, 91, 66]`, and remove it before committing.

Both: wire your feature into the menu. Each student adds their option as **option 1**, on the line directly below the menu title, and adds the matching branch directly below the `if choice == "0":` block. Example for the Maintainer:

```
def show_menu():
    print("\n=== Grade Book ===")
    print("1. Add student")
    print("0. Exit")
```

```
        if choice == "0":
            print("Goodbye!")
            break
        elif choice == "1":
            add_student()
```

The Collaborator does the same with `1. Show grades` and `show_grades()`. Both editing the same lines is deliberate: it sets up the merge conflict in Phase 5.

Commit in small steps (at least two commits each: one for the function, one for the menu), with short messages in the imperative ("Add show_grades function", not "changes" or "done"). Then push the branch:

```
git add grade_book.py
git commit -m "Add add_student function"
# ...second change...
git commit -am "Add 'Add student' option to menu"
git push -u origin feature/add-student
```

> Checkpoint 3: both branches appear on GitHub, each with two or more commits.

Each student runs `git log --oneline` on their own feature branch.

Maintainer (`feature/add-student`):

```
c4d5e6f (HEAD -> feature/add-student, origin/feature/add-student) Add 'Add
student' option to menu
a9b8c7d Add add_student function
b7e3a1c (origin/main, origin/HEAD, main) Add starter grade book with menu
loop
8743cf3 Initial commit
```

Collaborator (`feature/show-grades`):

```
5a6b7c8 (HEAD -> feature/show-grades, origin/feature/show-grades) Add 'Show
grades' option to menu
4f3e2d1 Add show_grades function
b7e3a1c (origin/main, origin/HEAD, main) Add starter grade book with menu
loop
8743cf3 Initial commit
```

Each branch has its own two commits on top of the same starter commit. Neither student can see the other's work yet.

Phase 4: Pull request and code review

The Maintainer's feature goes first.

1. **Maintainer** opens a pull request: base `main` ← compare `feature/add-student`. Give it a clear title and write `Closes #1` in the description.
2. **Collaborator** reviews it under **Files changed**: leave at least one inline comment, then submit the review as **Request changes**. A useful point to raise: the program crashes if marks are not numbers.
3. **Maintainer** fixes the issue on the same branch (for example, wrap the conversion in `try` / `except ValueError`), commits, and pushes. The pull request updates automatically.
4. **Collaborator** re-reviews and **Approves**.
5. **Maintainer** merges with **Create a merge commit**, then deletes the branch. Issue #1 closes automatically.

> Checkpoint 4: PR merged, Issue #1 closed, review comments visible in the PR history.

The Maintainer runs `git checkout main`, `git pull origin main`, then `git log --oneline`. Expected output:

```
e1f2a3b (HEAD -> main, origin/main, origin/HEAD) Merge pull request #4 from
<username>/feature/add-student
d3e4f5a Handle non-numeric marks in add_student
c4d5e6f Add 'Add student' option to menu
a9b8c7d Add add_student function
b7e3a1c Add starter grade book with menu loop
8743cf3 Initial commit
```

The merge commit at the top shows the pull request reached `main`. The PR is #4 because issues #1 to #3 share the same numbering.

Phase 5: Resolve a merge conflict

1. **Collaborator** opens a pull request for `feature/show-grades` with `Closes #2`. GitHub reports that the branch has conflicts, because `main` changed the same menu lines.
2. Bring the latest `main` into the feature branch locally:

```
git checkout main
git pull origin main
git checkout feature/show-grades
git merge main
```

Git stops with `CONFLICT (content): Merge conflict in grade_book.py`. The file now contains markers like:

```
<<<<<<< HEAD
    print("1. Show grades")
=======
    print("1. Add student")
>>>>>>> main
```

3. Edit the file so **both** features survive: make Add student option 1 and Show grades option 2, in both the menu and the `elif` branches. Delete every `<<<<<<<`, `=======` and `>>>>>>>` line.
4. Run the program and test both options together before committing.
5. Finish the merge and push:

```
git add grade_book.py
git commit -m "Merge main into feature/show-grades and resolve menu conflict"
git push
```

6. **Maintainer** reviews and approves; the **Collaborator** merges. Issue #2 closes.

> Checkpoint 5: `main` has both features, and the program runs with options 0, 1 and 2.

The Collaborator runs `git checkout main`, `git pull origin main`, then `git log --oneline`. Expected output:

```
f6a7b8c (HEAD -> main, origin/main, origin/HEAD) Merge pull request #5 from
<username>/feature/show-grades
9c8d7e6 Merge main into feature/show-grades and resolve menu conflict
e1f2a3b Merge pull request #4 from <username>/feature/add-student
d3e4f5a Handle non-numeric marks in add_student
5a6b7c8 Add 'Show grades' option to menu
c4d5e6f Add 'Add student' option to menu
4f3e2d1 Add show_grades function
a9b8c7d Add add_student function
b7e3a1c Add starter grade book with menu loop
8743cf3 Initial commit
```

All commits from both students now appear on `main`. The middle commits are listed by the time they were made, so their order may differ slightly from this example.

Phase 6: Sync and release

1. **Both** update their local `main` and look at the history:

```
git checkout main
git pull origin main
git log --oneline --graph --all
```

2. **Maintainer** publishes a release: **Releases → Draft a new release**, create tag `v1.0`, title it "Grade Book v1.0", click **Generate release notes**, and publish.

> Checkpoint 6: release v1.0 visible on the repository page; both local copies match GitHub.

Both students run `git pull origin main --tags`, then `git log --oneline`. The top line should now carry the release tag:

```
f6a7b8c (HEAD -> main, tag: v1.0, origin/main, origin/HEAD) Merge pull
request #5 from <username>/feature/show-grades
9c8d7e6 Merge main into feature/show-grades and resolve menu conflict
e1f2a3b Merge pull request #4 from <username>/feature/add-student
...
8743cf3 Initial commit
```

Both laptops should show exactly the same list as each other.

Bonus tasks

For pairs who finish early (about 20 min):

1. **Swap roles on Issue #3.** The Collaborator assigns themselves to "Show the class average" and follows the full branch → PR → review → merge cycle, with the Maintainer reviewing.
2. **Write a proper README.** Describe what the program does, how to run it, and who contributed, through a PR of course.

Command cheat sheet

Command	What it does
`git --version`	Check that Git is installed and show its version
`git config --global user.name "Your Name"`	Set the name recorded on all your commits

Command	What it does
`git config --global user.email "you@example.com"`	Set the email recorded on all your commits
`git config user.name` / `git config user.email`	Show the name or email Git is using in the current folder
`git clone <url>`	Download a copy of a remote repository
`cd <folder>`	Move into a folder, e.g. `cd grade-book`
`git status`	Show changed, staged and untracked files
`git checkout <branch>`	Switch to an existing branch, e.g. `git checkout main`
`git checkout -b <branch>`	Create a new branch and switch to it
`git add <file>`	Stage a file for the next commit
`git commit -m "message"`	Save staged changes as a commit
`git commit -am "message"`	Stage all tracked changes and commit in one step
`git push origin main`	Upload commits on `main` to GitHub (used only once, for the starter code)
`git push -u origin <branch>`	Upload a new branch and link it to the remote
`git push`	Upload new commits on a branch that is already linked
`git pull origin main`	Fetch and merge the latest `main` from GitHub
`git pull origin main --tags`	Fetch the latest `main` along with any release tags
`git merge main`	Merge `main` into the current branch
`git log --oneline`	Show the commit history, one line per commit
`git log --oneline --graph --all`	Show the history of all branches as a graph
`git diff`	Show unstaged changes line by line
`git revert <hash>`	Create a new commit that undoes an earlier one

Command	What it does
`git branch -d <branch>`	Delete a local branch that has been merged
`python grade_book.py`	Run the Grade Book program